SVKM’s NMIMS

Mukesh Patel School of Technology Management & Engineering /
School of Technology Management & Engineering

Year: II **Program:** B. Tech/MBA Tech **Semester:** IV

Subject: Object-Oriented Programming through JAVA

Smart City Traffic Management Simulator

Team Details

Name	Anupam Kanoongo	Arjun Borkar	Abhinav Jain
Roll No	F044	F005	F038
SAP ID	70022400311	70022400057	70022400303

Bachelor of Technology (Computer Engineering)
Second Year, Semester IV

Faculty: Dr. Nirmal Gupta
AY 2025-26

INDEX

1 Introduction **4**

1.1 Background and Significance 4

1.2 Need for the Project 4

1.3 Scope and Boundaries 4

1.4 Objectives 4

1.5 Report Roadmap 5

2 Requirement Analysis **5**

2.1 Functional Requirements 5

2.2 Non-Functional Requirements 6

2.3 User Stories and Use Cases 6

2.4 Assumptions, Dependencies, and Exclusions 7

2.5 Requirement Traceability Summary 7

3 Development Technologies **7**

3.1 Language and Framework 7

3.2 Technology Comparison: Swing vs JavaFX 8

3.3 Build and Deployment Tools 8

4 Modelling Classes **8**

4.1 Class Design Overview 8

4.2 Key Classes and Responsibilities 9

4.3 Object Interaction Scenario: Ambulance Preemption 10

4.4 Refactoring Notes 11

5 Progress Review (Week 10/11) **11**

5.1 Milestone Snapshot 11

5.2 Planned vs Completed Tasks 12

5.3 Risk Register 12

5.4 Blockers and Corrective Actions 12
5.5 Action Plan for the Upcoming Week 13

6 Implementation and Output **13**
6.1 System Architecture and Execution Flow 13
6.2 Key Algorithmic Decisions 13
6.3 Testing Summary 14
6.4 Output Snapshots 15
6.5 Deployment and Packaging Notes 17

7 Observations and Learnings **17**
7.1 Technical Learnings 17
7.2 Process and Team Learnings 18

8 Future Scope **19**
8.1 Short-Term Enhancements (1-3 Months) 19
8.2 Medium-Term Enhancements (3-6 Months) 19
8.3 Long-Term Vision (1+ Years) 19

9 Conclusion **20**

A Glossary **21**

B Abbreviations **21**

C Bibliography and References **21**

Abstract

The issue of urban traffic congestion and delayed emergency vehicle responses plays a key role in current intelligent infrastructure design and development. Modern traffic light coordination operates under fixed-time traffic light signal cycles when the allocation of the green light for any lane is done without consideration of whether that road has any traffic passing by or not. Not only the current signaling mechanism does not allocate green time effectively but, additionally, it does not contain any mechanisms for prioritizing any type of vehicle in the case if it was necessary to provide special treatment for it. This paper describes a traffic simulator developed for a smart city using Java programming language based on the object-oriented programming paradigm. The application enables the detection of empty lanes and allocating the saved-up green time to congested lanes through the so-called Time Bank approach. Additionally, the traffic signal system can allow an emergency vehicle, such as an ambulance, to have the right of way through special algorithmic mechanisms. The application, which is written solely based on Java Swing components, illustrates practical implementation of major concepts of OOP.

1 Introduction

1.1 Background and Significance

Due to urban sprawl, there has been an increase in the number of vehicles using the roads in the cities. Intersections are the weakest parts of the cities, and regular traffic lights have made this problem even worse. Regular traffic lights work on a timer, which cannot be modified based on the traffic. As a result, green time is wasted at intersections where there are no cars but in other intersections where cars are stuck in traffic. Thus, more carbon dioxide and fuel are consumed by cars.

1.2 Need for the Project

Even though the complex ATSC models have been used by many major cities around the world, such models utilize expensive sensors and sophisticated machine learning algorithms. There arises a great need to create a lightweight and rule-based model of intelligent behavior that will aid in understanding such intelligence from a scientific point of view. This has been achieved in the current project through designing a virtual environment of traffic control simulation in which users are allowed to explore cause-and-effect relationships in traffic control. Besides, this project has also been designed as an academically difficult problem that aids in learning OOP principles.

1.3 Scope and Boundaries

In this particular study, the scope of the project will only be limited to one particular four-way intersection. In this simulation, there are four types of vehicle classes that have been used (Private Car, Public Bus, Motorcycle, and Ambulance), each having their own particular driving characteristics. The scope of this model includes lane based queuing, transition, traffic data statistics and logging.

1.4 Objectives

Primary Objectives:

- Develop an adaptive traffic light control algorithm that changes lights ahead if there are no vehicles in the active lane, making use of any saved time in the process.
- Devise an emergency control mechanism that immediately overrules the usual traffic light system and prioritizes ambulances over everything else.

- Meet the demands of the OOPJ class by fully incorporating the concepts of abstraction, inheritance, polymorphism, encapsulation, and collection.

Secondary Objectives:

- Design a Java Swing graphical user interface that enables pausing/resuming tasks and regulates the rate.
- Ensure high-quality software through exception handling and session logging.
- Distribute the application through executable files (.exe, .dmg, .deb, .rpm) via GitHub Actions CI/CD.

1.5 Report Roadmap

The remaining sections in this report are structured as follows: Section 2 discusses Requirement Analysis, which includes Functional and Non-Functional Requirements Specification. Technology selection justification is provided in Section 3. Section 4 explains class modeling and architecture design. A realistic project progress summary for weeks 10/11 is presented in Section 5. Sections 6 outlines the implementation and testing process, and results. Lessons learnt are presented in Section 7. Future work scope is discussed in Section 8.

2 Requirement Analysis

2.1 Functional Requirements

FR-ID	Requirement Statement	Priority	Rationale
FR-01	The system must allow for the creation of various vehicle types (Car, Bus, Motorcycle) in different lanes.	High	Core interaction is necessary for simulating the system.
FR-02	The system must offer Smart Mode, where the green signal turns on sooner if the lane is empty for 3 seconds.	High	It is one of the core functionalities showing adaptiveness of traffic control.
FR-03	The system must incorporate the Time Bank feature, which saves time in empty lanes and assigns saved time to congested lanes.	Medium	Ensures fairness of the operation.

FR-ID	Requirement Statement	Priority	Rationale
FR-04	The system must create an ambulance that overrides the current signal cycle and gets a green signal.	High	It is one of the core functionalities to demonstrate the preemption feature.
FR-05	The GUI must display certain statistics (cars per lane, cycle timer and time bank).	Medium	Necessary for visualization of the background operations.
FR-06	The system should log all the significant events to the text files.	Low	Meets the requirement for file handling.

2.2 Non-Functional Requirements

NFR-ID	Requirement Statement	Measurable Target	Validation Method
GUI Rendering	It must have no tearing or glitches during operation.	It must have at least 20 FPS (refreshing period is 50 milliseconds).	Logging of observations and timer results.
Simulation Logic	The logic of the simulation should not depend on the speed of GUI rendering.	The logic timer should work one time per second (one millisecond).	Code review and checking its runtime output.
Java Runtime Environment	There should be no need for installation of the Java Runtime Environment on your computer.	Bundled JRE size must be less than 100 megabytes.	Fresh VM testing.
Input Error Handling	The program will not crash on any invalid user input.	There should be no exceptions that are not caught in runtime during normal use.	Checking speed controls and spawning creatures boundaries.

2.3 User Stories and Use Cases

- **As a student user**, I should have an option to pause and play the simulation for examining the situation at the intersection and the values in the time bank.
- **As a traffic analyst**, I should be able to toggle between Smart Mode and Manual Mode to compare the effectiveness of fixed-time and adaptive signal timing schemes.

- **As an instructor**, I should have the ability to check application logs to ensure the sequence of actions and proper triggering of emergency override functionality.

2.4 Assumptions, Dependencies, and Exclusions

Assumptions: There is no turning done by any vehicle, and it goes from one side to the other in a straight path. Every vehicle moves at an equal acceleration, which varies depending on the type of vehicle.
Dependents: The continuous integration build pipeline depends on the use of GitHub Actions, as well as `jpackage`, the chain of tools provided by JDK 21.
Exclusions: Real-world rules regarding collisions, weather, mapping, and pedestrian crossing have been explicitly excluded.

2.5 Requirement Traceability Summary

- **FR-01, FR-04** $\rightarrow$ Implemented in the `Vehicle` abstract hierarchy and the `TrafficSimulatorGUI` spawn action listeners.
- **FR-02, FR-03** $\rightarrow$ Implemented in `TrafficController.checkSmartRedistribution()` and time bank logic.
- **FR-06** $\rightarrow$ Implemented via the custom `SimulatorLogger` stream redirection.

3 Development Technologies

The technology stack was selected to satisfy the academic constraints of the OOPJ curriculum while utilizing modern software engineering practices for deployment.

3.1 Language and Framework

Java (requires JDK 8 run-time minimum, min. JDK 21 for CI/CD): The choice was made based on the fact that Java is a language which needs to be used as per the course requirement. Java provides excellent support for object-oriented programming along with typing and portability features.
Java Swing: Used for developing the GUI part of the project.

Reasoning: While JavaFX could provide additional capabilities to the application, the Java SE API already provides Swing. Hence, using Swing helped avoid using any third-party dependencies associated with JavaFX using technologies like Maven or Gradle.

3.2 Technology Comparison: Swing vs JavaFX

Criteria	Java Swing	JavaFX
Dependency	Built-in (Standard JDK)	Requires external modules (since JDK 11)
Architecture	MVC-based, imperative	Scene Graph, declarative (FXML)
Styling	LookAndFeel (Custom Java code)	CSS-based styling
Decision	**Selected.** Lower overhead for a simple 2D canvas simulation.	Rejected. Too complex to bundle natively without external build tools.

3.3 Build and Deployment Tools

GitHub Actions (CI/CD): Involved in the automated compilation, packaging, and deployment process. Whenever an update is made to the main branch, a matrix builds on macOS, Windows, and Ubuntu virtual machines.
jpackage (JDK Utility): Involved in creating a standalone application from the JAR file with an appropriate Java Runtime Environment (JRE). Native bundles of files are produced (`.dmg`, `.exe`, `.deb`, `.rpm`).

Benefits: The target audience will not need to have Java installed for evaluation purposes. Thus, there will be no "it works on my machine" problems resulting from Java version differences and environment variables.

4 Modelling Classes

The system architecture is heavily reliant on class hierarchies and interfaces to decouple the simulation engine from the GUI rendering logic.

4.1 Class Design Overview

Domain Model has an abstract base class for the entities, an interface for the special abilities and a controller that acts as the brain of the game. This design pattern offers easy extension, meaning that adding another vehicle such as FireTruck will require only implementing EmergencyVehicle interface and subclassing the Vehicle class without modifying `TrafficController` core logic.

4.2 Key Classes and Responsibilities

- `Vehicle` **(Abstract Class)**
  - *Attributes:* `protected int x, y, speed`, `Color color`, `String laneID`.
  - *Methods:* `abstract void move(String signalState)`, `boolean isInIntersection()`.
  - *Responsibility:* Defines the blueprint and shared spatial logic for all vehicles.

Listing 1: Vehicle.java (Abstract Base Class)

```java
public abstract class Vehicle {
    protected int x, y, speed;
    protected Color color;
    protected String laneID;
    protected boolean isMoving;

    public abstract void move(String signalState);

    protected boolean isInIntersection() {
        return x >= (INTERSECTION_CENTER_X - INTERSECTION_HALF_WIDTH) &&
               x <= (INTERSECTION_CENTER_X + INTERSECTION_HALF_WIDTH) &&
               y >= (INTERSECTION_CENTER_Y - INTERSECTION_HALF_WIDTH) &&
               y <= (INTERSECTION_CENTER_Y + INTERSECTION_HALF_WIDTH);
    }
}
```

- `EmergencyVehicle` **(Interface)**
  - *Methods:* `boolean isOnEmergency()`, `void setOnEmergency(boolean status)`.
  - *Responsibility:* Provides a contract for vehicles that can bypass standard traffic laws.

Listing 2: EmergencyVehicle.java and Ambulance.java snippet

```java
public interface EmergencyVehicle {
    boolean isOnEmergency();
    void setOnEmergency(boolean status);
}

public class Ambulance extends Vehicle implements EmergencyVehicle {
    private boolean emergencyMode = true;

    @Override
    public void move(String signalState) {
        if (isInIntersection()) {
            isMoving = true;
            moveInLaneDirection(speed);
            return;
        }
        boolean approachingRed = "RED".equals(signalState) &&
            isApproachingIntersection();
        if (approachingRed && emergencyMode) {
```

```
            isMoving = true; // Never fully stops at red
            moveInLaneDirection(speed / 2);
        } else {
            // ... normal movement
        }
    }
}
```

- **`TrafficController`**
  - *Attributes:* `Map<String, TrafficLight> trafficLights`, `int savedTimeBank`.
  - *Methods:* `void update()`, `void checkSmartRedistribution()`, `void checkEmergencyOverride()`.
  - *Responsibility:* Manages the state machine of the intersection, handles cycle ticks, triggers early lane switches, and processes ambulance preemption.

Listing 3: TrafficController.java snippet

```
public class TrafficController {
    private Map<String, TrafficLight> trafficLights;
    private int savedTimeBank;
    // ...
    public void update() {
        // Tick light, check emergencies, process transitions
    }
}
```

- **`SimulatorLogger`**
  - *Attributes:* `CopyOnWriteArrayList<String> buffer`, `BufferedWriter fileWriter`.
  - *Responsibility:* Intercepts `System.out` via a custom `TeePrintStream` to provide timestamped file logging and an in-memory ring buffer for the GUI.

4.3 Object Interaction Scenario: Ambulance Preemption

1. User clicks "Spawn Ambulance" in the GUI.
2. `TrafficSimulatorGUI` instantiates `new Ambulance(x, y, "WEST")` and adds it to the `vehicles` ArrayList.
3. The 1-second `updateTimer` triggers `TrafficController.update()`.
4. `TrafficController` calls `checkEmergencyOverride()`, iterating through the vehicles collection.

Listing 4: TrafficController.java - checkEmergencyOverride()

```
private void checkEmergencyOverride() {
    for (Vehicle vehicle : vehicles) {
```

```java
        if (vehicle instanceof Ambulance && !vehicle.hasPassedIntersection()) {
            String ambulanceLane = vehicle.getLaneID();
            if (!ambulanceLane.equals(getCurrentLane())) {
                TrafficLight currentLight = trafficLights.get(getCurrentLane());
                if (currentLight.isGreen() && currentLight.getTimeRemaining() > 3) {
                    if (ambulanceLane.equals(getNextLane())) {
                        currentLight.activateYellow();
                    } else {
                        currentLight.setTimeRemaining(3);
                    }
                }
            }
            break;
        }
    }
}
```

5. It detects the `Ambulance` using the `instanceof` operator (or interface check).
6. It queries the current active lane. Since "WEST" is currently RED, it commands the active `TrafficLight` to `activateYellow()` immediately, shortening its cycle to 3 seconds.
7. In the subsequent ticks, the system cycles to "WEST", turning it GREEN, allowing the `Ambulance` to clear the intersection.

4.4 Refactoring Notes

In the early stages of development, the GUI had `switch` blocks to decide how to render and control the behavior of each vehicle. This was optimized by having the behavior code reside inside the `Vehicle` class (like `Motorcycle`, which overrides the `move()` method to plow through yellow lights). This resulted in complete polymorphism since the GUI only calls `vehicle.move(signalState)`.

5 Progress Review (Week 10/11)

This is the status of the project after the end of Week 11, when the project shifts from focusing on implementing core logic to integration and stability.

5.1 Milestone Snapshot

- **Done:** Hierarchy of vehicles through OOP, basic rendering using Swing, fixed-timed signals algorithm, and exception handling.
- **WIP:** Tweaking of smart mode algorithm, and CI/CD pipelines (`jpackage`) configuration.
- **Not done:** Documenting, testing of edge cases, preparing for final presentation.

5.2 Planned vs Completed Tasks

Task	Plan Date	Actual	Status	Remarks
Base Vehicle classes	Wk 8	Wk 8	Done	Implemented smoothly.
Swing GUI Layout	Wk 9	Wk 9	Done	Coordinated and mapped to 1080p screens.
Fixed Cycle Controller	Wk 9	Wk 10	Done	Delayed due to timer sync issues between GUI and Logic.
Smart Mode (Time Bank)	Wk 10	Wk 11	Done	Required major logic refactor for real-time polling.
Emergency Override	Wk 11	Wk 11	Done	Interfaced correctly with Ambulance class.
CI/CD Packaging	Wk 11	–	In Prog	Struggling with WiX toolset paths on Windows runners.

5.3 Risk Register

Risk	Impact	Prob.	Mitigation Strategy	Status
Swing Concurrency Exception	High	High	Iterate lists backwards or use `CopyOnWriteArrayList` for the vehicle collections.	Resolved
`jpackage` fails on Windows runner	Medium	High	Dynamically locate WiX toolset path in GitHub Actions YAML via PowerShell scripts.	Active
Time Bank overflows	Low	Medium	Cap the time bank at 40s to prevent infinite green loops that starve other lanes.	Resolved

5.4 Blockers and Corrective Actions

Bug: Occasionally, there would be an unexpected event where the exception of `ConcurrentModificationExc` would happen in the GUI source code, as a result of the deletion of objects inside the vehicles ArrayList from the timer source code, once the objects have moved beyond the screen's border.

Fix: The code for `updateVehicles()` for loop inside the GUI class has been changed to the reverse order (`for (int i = vehicles.size() - 1; i >= 0; i--)`).

5.5 Action Plan for the Upcoming Week

1. Fix the bug with the Windows `.exe` installer in the GitHub Actions pipeline.
2. Conduct an end-to-end test of the entire system based on all the grading requirements.
3. Get ready to compile the `report.tex` file.

6 Implementation and Output

6.1 System Architecture and Execution Flow

A separation between the presentation and control layers through a timer-based mechanism is used in the system architecture. This is essential in designing simulations.

- **Presentation Layer:** Managed by `TrafficSimulatorGUI`. `javax.swing.Timer` runs with a minimum period of 50ms (20 FPS) for `vehicle.move()` and `intersectionPanel.repaint()` to execute.
- **Control Layer:** Managed by `TrafficController`. The second timer initiates events precisely after 1000ms (1Hz), which serves to tick traffic lights, check if smart mode conditions are met, handle any override commands, and update time bank.

As such, the ability to increase or decrease the animation rate (by moving the speed slider on the interface) leads to an alteration of the 50ms delay in the animation loop while maintaining the 1 second delay needed for realistic traffic light timers.

6.2 Key Algorithmic Decisions

The "Smart Redistribution" algorithm relies on continuous real-time lane polling. When the 1-second control timer ticks, it checks:
`if (!hasVehicles && elapsedGreenTime >= minimumGreenTime && timeRemaining > 1)`
If true, it calculates the `remainingTime`, adds it to the global `savedTimeBank`, and forces an immediate yellow transition.

Listing 5: TrafficController.java - checkSmartRedistribution()

```
private void checkSmartRedistribution() {
    String currentLane = getCurrentLane();
    TrafficLight currentLight = trafficLights.get(currentLane);
    boolean hasVehicles = hasVehiclesInLane(currentLane);
```

```
    int minimumGreenTime = 2;
    int elapsedGreenTime = currentLight.getGreenTime() - currentLight.getTimeRemaining();

    if (!hasVehicles && elapsedGreenTime >= minimumGreenTime && currentLight.
        getTimeRemaining() > 1) {
        int remainingTime = currentLight.getTimeRemaining();
        savedTimeBank += remainingTime;
        currentLight.activateYellow();
    }
}
```

When advancing to the next lane, the algorithm allocates a base green time (20s) plus a calculated bonus: `Math.min(savedTimeBank, vehicleCount * 3)`. This bounds the maximum extension to 3 seconds per waiting vehicle, capping at a hard limit of 40 seconds total.

Listing 6: TrafficController.java - advanceToNextLane()

```
int greenTime;
if (nextLaneHasVehicles && savedTimeBank > 0) {
    int vehicleCount = countVehiclesInLane(nextLane);
    int bonusTime = Math.min(savedTimeBank, vehicleCount * 3); // max 3s per vehicle
    int maxGreen = 40; // cap at 40s to maintain cycle balance
    greenTime = Math.min(baseGreenTime + bonusTime, maxGreen);
    int usedTime = greenTime - baseGreenTime;
    savedTimeBank -= usedTime;
}
```

6.3 Testing Summary

Testing was conducted manually through structured test scenarios covering standard operations and boundary conditions.

Test ID	Scenario / Input	Expected Output	Result
TC-01	Spawn 1 Car in NORTH. Wait for cycle.	Car stops at RED, moves on GREEN, clears intersection.	Pass
TC-02	Toggle Smart Mode ON. Spawn 0 cars in active lane.	Lane switches early; time bank increases on UI.	Pass
TC-03	Spawn Ambulance while lane is RED.	Active lane forces YELLOW, ambulance lane goes GREEN.	Pass
TC-04	Pause simulation, spawn vehicles, resume.	New vehicles appear frozen, move only upon resume.	Pass

Test ID	Scenario / Input	Expected Output	Result
TC-05	Speed slider set to 4x.	Animation moves fast; cycle timers tick at normal 1s rate.	Pass

6.4 Output Snapshots

The output screen after running Swing provides an aerial view of the intersection with roads drawn in dark grey colors and yellow dashed lines demarcating the lanes. Once the cars start moving, the boxes appear on the lanes.

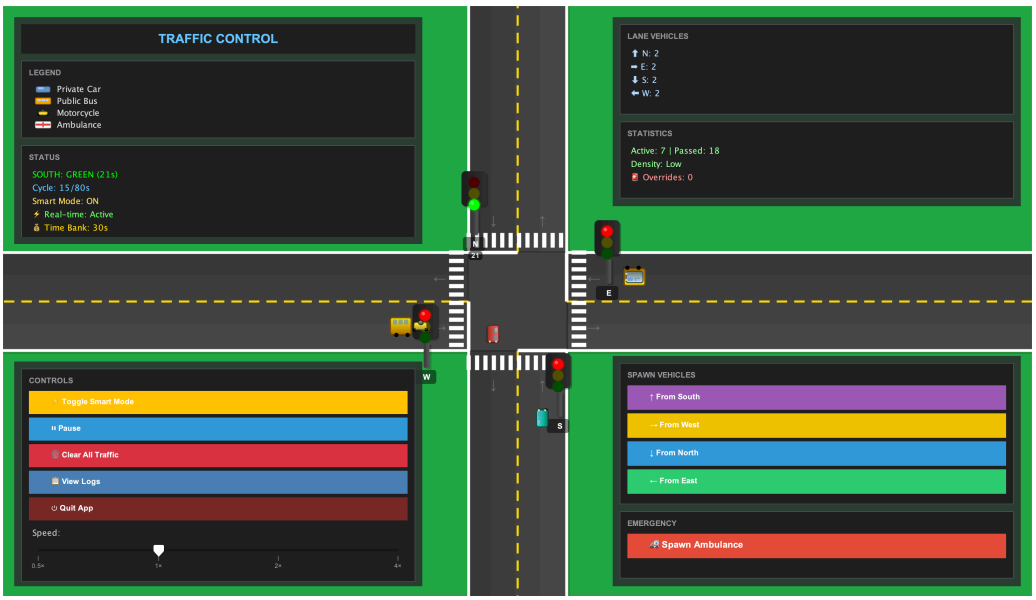

Figure 1: Main Simulation Window

- **Trigger for Ambulance:** When the Ambulance (Red) enters the system, the boundary starts to glow. The current green light turns into a yellow light with a period of 3 seconds, and the following entry is made in the terminal: `[1712761301234] Emergency Override: Ambulance in WEST`

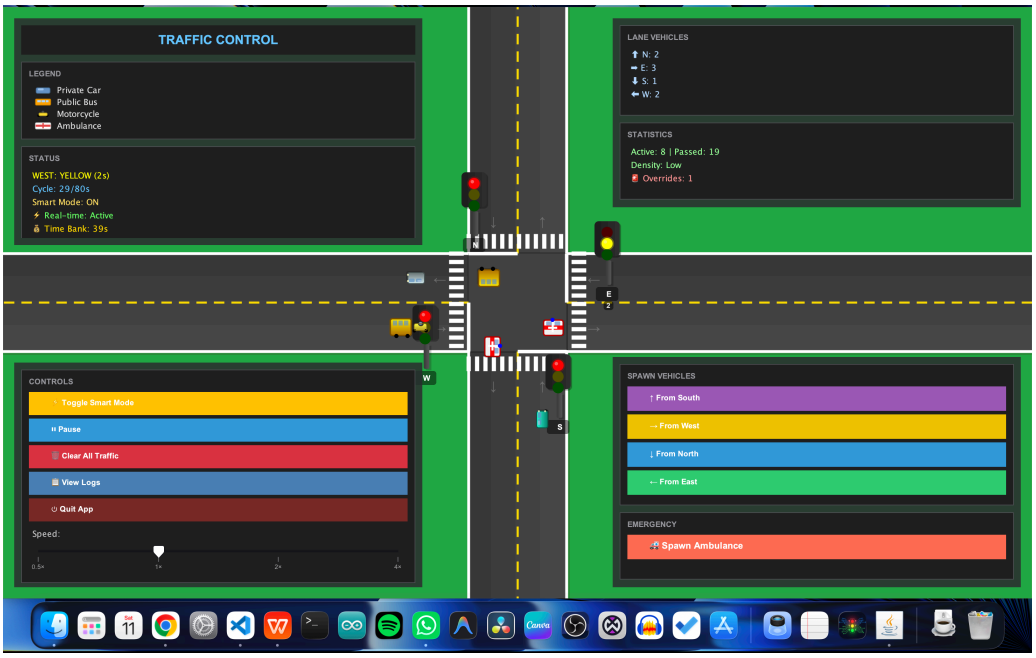

Figure 2: Emergency Override Triggered

- **Operation in Smart Mode:** The label on the GUI for Time Bank updates dynamically (for example, `Time Bank: 14s`). In case of an overloaded lane receiving a green light, the countdown label starts counting dynamically from a specific number (such as `NORTH: GREEN (34s)` instead of the usual 20s).

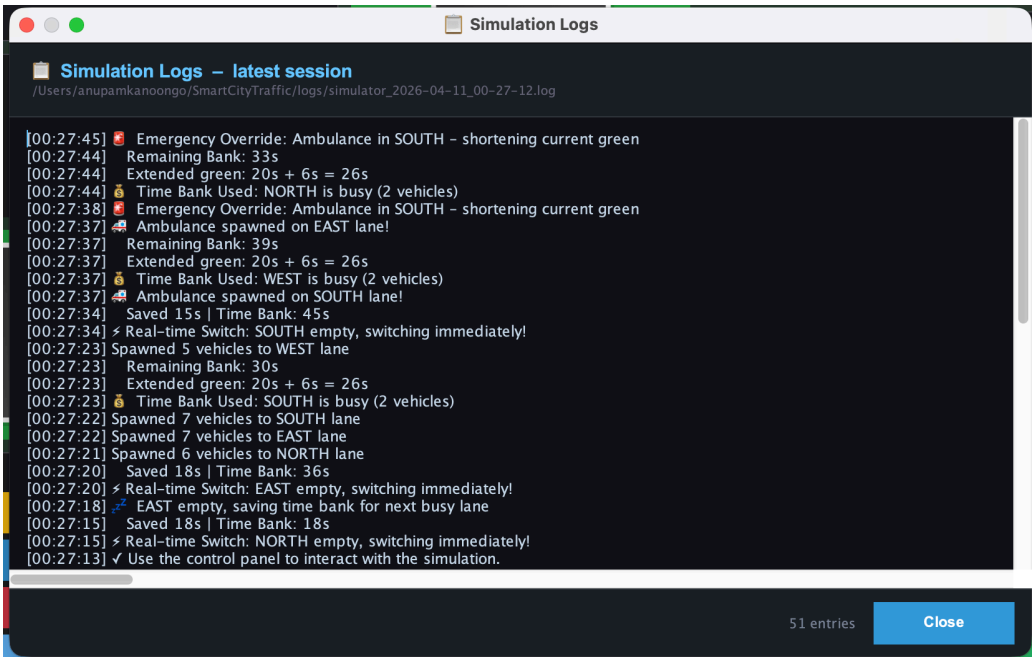

Figure 3: Simulator Log Viewer

6.5 Deployment and Packaging Notes

This project is well-positioned to make good use of GitHub Actions. The `build-installers.yml` pipeline utilizes `jpackage`, which is part of the JDK 21 release, to convert the `SmartCityTraffic.jar` that has been generated into an installable native application. In doing so, the result is an `.exe` installation package for Windows, a `.dmg` installation package for macOS, and `.deb`

7 Observations and Learnings

7.1 Technical Learnings

- **Polymorphism reduces complexity:** Through the implementation of the polymorphic move(String signalState) function within the Vehicle class, the main loop for the simulation program becomes simple, as seen below: `for(Vehicle v:vehicles){v.move(signal);}`, effectively making it unnecessary to write difficult and messy code through the `if-else` or `switch` statement to distinguish between

Listing 7: TrafficSimulatorGUI.java - Polymorphic vehicle loop

```
private void updateVehicles() {
    for (int i = vehicles.size() - 1; i >= 0; i--) {
        Vehicle vehicle = vehicles.get(i);
        String laneID = vehicle.getLaneID();
        TrafficLight laneLight = trafficController.getTrafficLight(laneID);
        String signalState = laneLight != null ? laneLight.getCurrentState() : "RED"
            ;

        // Polymorphic call: Ambulance, Car, Bus move differently
        vehicle.move(signalState);
    }
}
```

- **Event Dispatch Thread (EDT):** It was discovered that the application experienced temporary freezes and display issues because the intensive computation was performed in the `main()` method. The problem was solved using `SwingUtilities.invokeLater()`.

Listing 8: Main.java - Safely starting the Swing GUI

```
public static void main(String[] args) {
    SimulatorLogger.install();
    SwingUtilities.invokeLater(() -> {
        TrafficSimulatorGUI gui = new TrafficSimulatorGUI();
        gui.setVisible(true);
    });
}
```

- **Log Redirection:** Implementing the This is how the `SimulatorLogger` showed us how to extend the simple `PrintStream`. Once we wrote our custom `TeePrintStream`, we could

catch all those calls to `System.out.println()` that went into printing both to the console and the file, while keeping the hundreds of lines of the logging code untouched.

Listing 9: SimulatorLogger.java - TeePrintStream

```java
private static final class TeePrintStream extends PrintStream {
    @Override
    public void println(String x) {
        super.println(x); // Print to original System.out
        capture(x == null ? "null" : x); // Route to our log file & UI buffer
    }

    private void capture(String line) {
        if (instance == null) return;
        String tagged = isErr ? "[ERR] " + line : line;
        instance.writeLine(tagged);
    }
}
```

7.2 Process and Team Learnings

- **Good things happened because:** Strict modularization of the work process. Since Arjun had already implemented the domain model (`Vehicle` inheritance) and Abhinav had made the control module (`TrafficController`), Anupam could work on the GUI alone, without anyone else interfering with his work.

- **Bad things happened because:** Using ZIP files in instant messaging software for the program integration caused many clashes and loss of logic. At the middle of week 4, we moved to Git and GitHub, which slowed us down slightly; yet, the move positively impacted our work greatly, as it stabilized our code and automated the CI/CD pipeline.

- **What was learned when working together (SSOT):** Initially, there were hard-coded parameters in many classes, specifying the lanes where vehicles were supposed to be placed. To overcome this issue, we centralized all the geometry parameters as the `protected static` fields of `Vehicle`, calculating these parameters dynamically through the screen size listener attached to the GUI.

8 Future Scope

This simulation model has an awesome underlying framework that can easily be adapted into a good traffic analysis tool.

8.1 Short-Term Enhancements (1-3 Months)

- **Persistence of Configuration Settings:** Introduce functionality for saving/loading simulation parameters (default green light period, threshold for smart mode activation, speeds) via JSON/Properties file formats.
- **Pedestrian Traffic Light Crossing:** Introduce the `Pedestrian` object type and include crossing of the intersection into simulation phases.
- *Effort/Value:* This effort would be relatively low while the value from the feature addition would be high.

8.2 Medium-Term Enhancements (3-6 Months)

- **Multi-intersection Network:** Expand the grid network such that 4 or more intersection points exist.
- **Green Wave Synchronization:** Formulate a method whereby the synchronization process among the adjacent intersections results in a group of vehicles encountering a series of green lights.
- *Value/Effort:* High effort. Requires refactoring the monolithic GUI into a scrollable viewport and making the `TrafficController` coordinate with peer controllers. Risk of severe performance degradation if Swing rendering is not heavily optimized.

8.3 Long-Term Vision (1+ Years)

- **AI/ML Density Prediction:** Replace the rule-based "Smart Mode" with a reinforcement learning model. The model could ingest historical log files generated by the `SimulatorLogger` to train an agent that optimizes signal timings based on time-of-day traffic patterns.
- **IoT Sensor Interface:** Expose a REST API that allows the simulation to ingest real-time queue data from physical camera sensors (using computer vision tools like OpenCV) deployed at actual intersections, turning the software into a live digital twin dashboard.

9 Conclusion

Each of the main and subsidiary objectives defined by the Smart City Traffic Simulator while analyzing requirements has been fulfilled by the system successfully. The addition of an adaptive traffic control algorithm allows the system to effectively highlight the advantages of lane polling as well as "Time Bank." These can be helpful in minimizing idle green light time and thus maximizing throughput. Additionally, the robust emergency preemption mechanism ensures quick aid for ambulances.

From an academic perspective, this can be considered an absolute implementation of OOP using Java programming language. The employment of abstract classes in vehicle hierarchy, interface for emergency operations, and polymorphism for method call has resulted in very neat and maintainable code. The separation between the timer for GUI visualization and the timer for actual logic has ensured smooth visualization without interfering with the determinism of the simulation process.

While the simulation is limited to one intersection right now and operates on logic rather than predictions, it has reached a very high level of software development. With exception handling, session logging, and CI/CD cross-platform package building present, the simulation turns from an average script into a real-life desktop application. The simulation serves as a strong foundation of technical capabilities for further expansion and inclusion of more nodes and AI modeling of cities.

A Glossary

- **Time Bank:** A conceptual storage mechanism in the controller that accumulates seconds saved when a green light is terminated early due to an empty lane.
- **Preemption:** The act of interrupting normal traffic signal operations to transfer right-of-way to an emergency vehicle.
- **Event Dispatch Thread (EDT):** A special background thread in Java used to process graphical user interface events and update Swing components.
- **Digital Twin:** A virtual representation that serves as the real-time digital counterpart of a physical object or process (e.g., a real-world intersection).

B Abbreviations

- **OOP / OOPJ:** Object-Oriented Programming (with Java)
- **GUI:** Graphical User Interface
- **ATSC:** Adaptive Traffic Signal Control
- **CI/CD:** Continuous Integration / Continuous Deployment
- **JRE:** Java Runtime Environment
- **MVC:** Model-View-Controller
- **AI/ML:** Artificial Intelligence / Machine Learning

C Bibliography and References

[1] Oracle Corporation. *Java Platform, Standard Edition 17 Documentation*. 2025. Available at: https://docs.oracle.com/en/java/javase/17/

[2] Oracle Corporation. *Java Swing Tutorial*. 2025. Available at: https://docs.oracle.com/javase/tutorial/uiswing/

[3] Oracle Corporation. *Java Collections Framework*. 2025. Available at: https://docs.oracle.com/javase/8/docs/technotes/guides/collections/

[4] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1st Edition, 1994.

[5] Sayed Z. Nekoo, et al. *Adaptive Traffic Signal Control: A Comprehensive Review*. IEEE Transactions on Intelligent Transportation Systems, Vol. 22, No. 8, 2021.